Exploiting SQL Injection Vulnerabilities on a Live Web Server

Internet and Web Application Security 3e - Lab 2

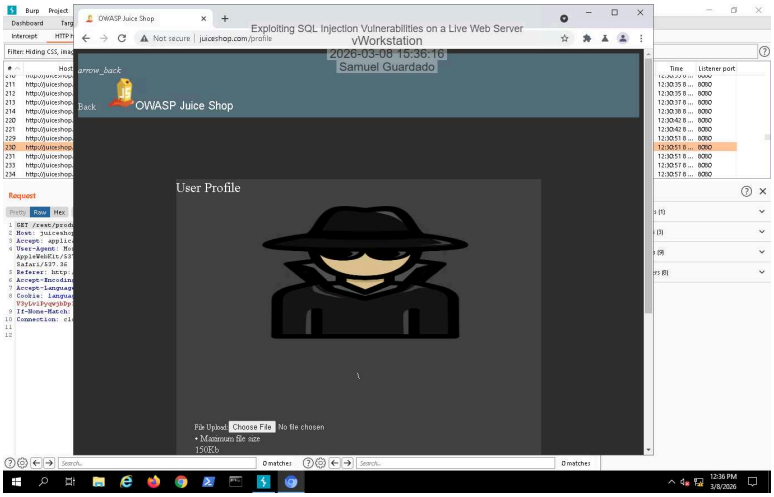
Student: Samuel Guardado
Email: sguar5@brockport.edu
Report Generated: Thursday, July 30, 2026 at 5:17 PM

Time on Task: 9 hours, 21 minutes
Progress: 100%

Section 1: Hands-On Demonstration

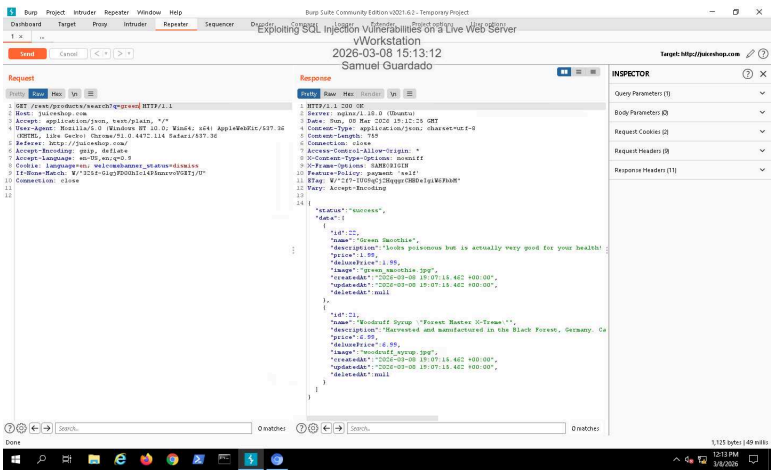
Part 1: Assess the Vulnerability of a Web Application Field

21. **Make a screen capture** showing the Administrator's user profile.

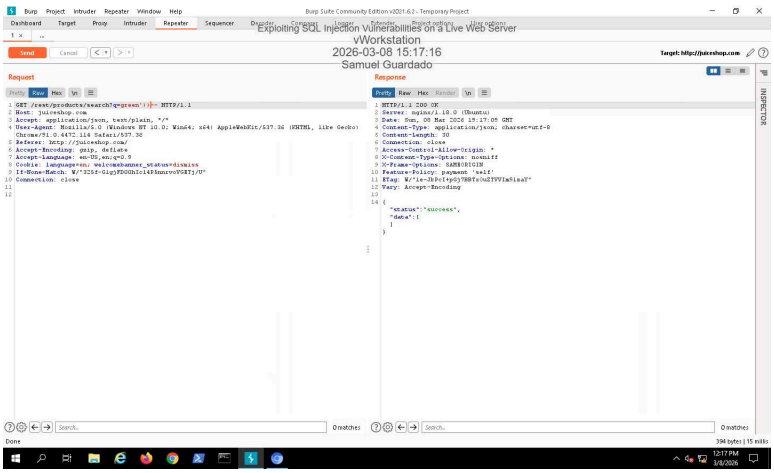


Part 2: Use Burp Suite to Identify the Data Structure of a Web Application

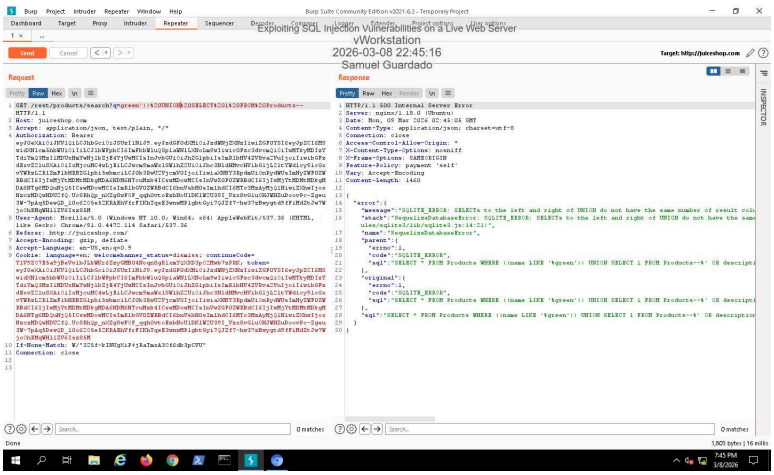
20. **Make a screen capture** showing the modified search in the Request panel and the JSON results with Green Smoothie and Woodruff Syrup in the Response panel.



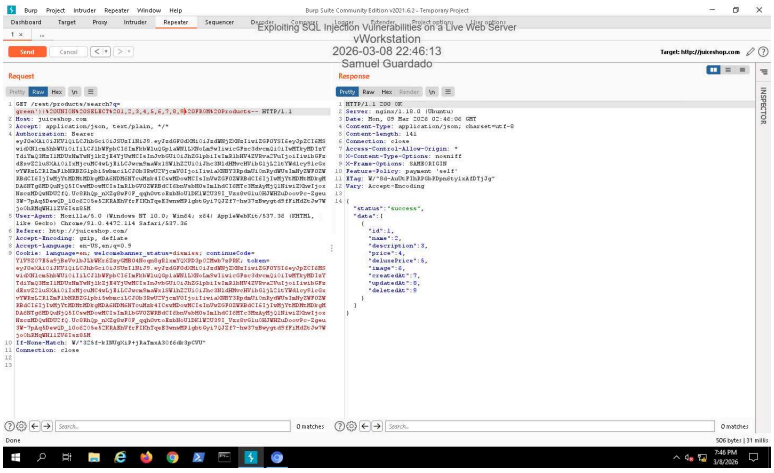
24. **Make a screen capture** showing the modified search in the Request panel and the JSON results that indicate a successful SQL injection.



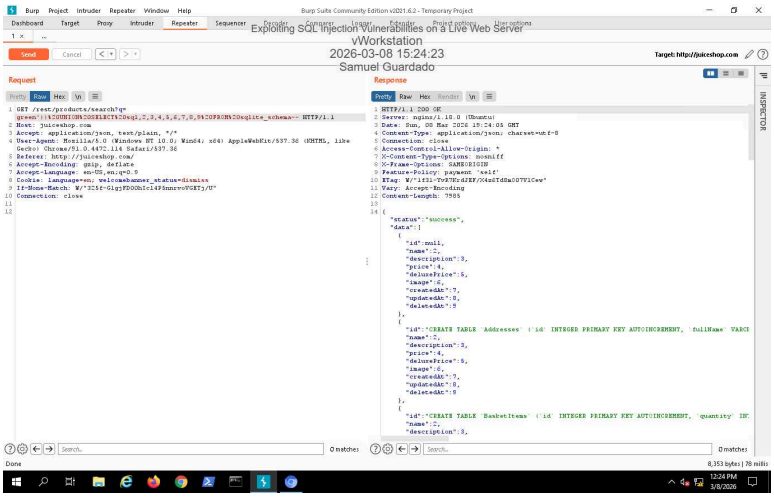
26. **Make a screen capture** showing the modified search in the Request panel and the JSON results that indicate an SQLite error.



29. **Make a screen capture** showing the modified search in the Request panel with the correct number of columns in the Products table and the JSON results that indicate a successful SQL injection.

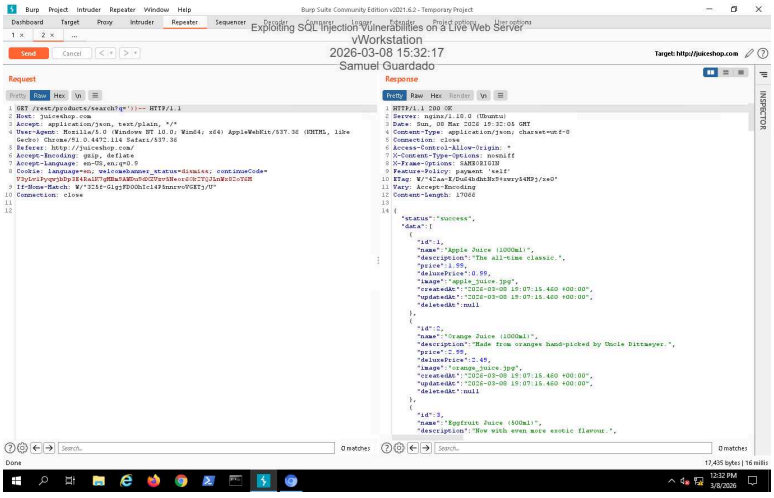


31. **Make a screen capture** showing the modified search in the Request panel and the JSON results that reveal the database schema in your successful SQL injection.

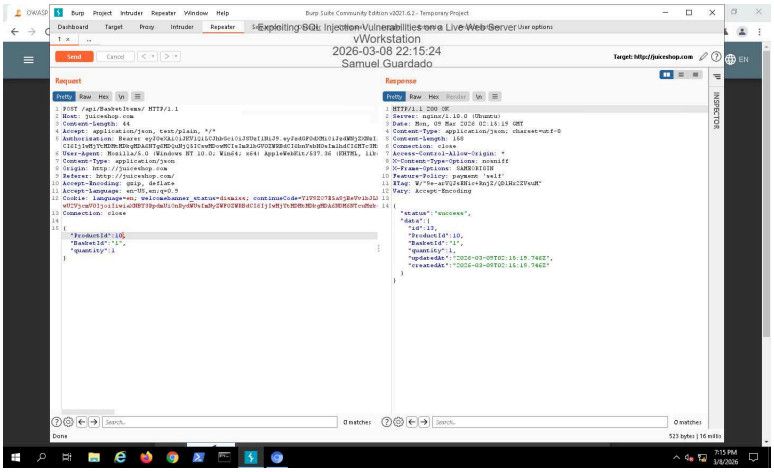


Part 3: Exploit Injection Vulnerabilities

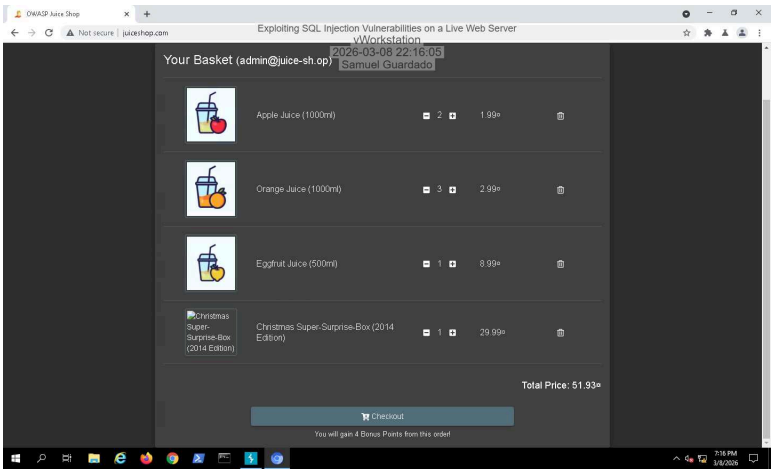
5. **Make a screen capture** showing the modified search in the Request panel and the JSON results that indicate a successful SQL injection.



27. **Make a screen capture** showing the modified search in the Request panel and the JSON results with the product ID change.



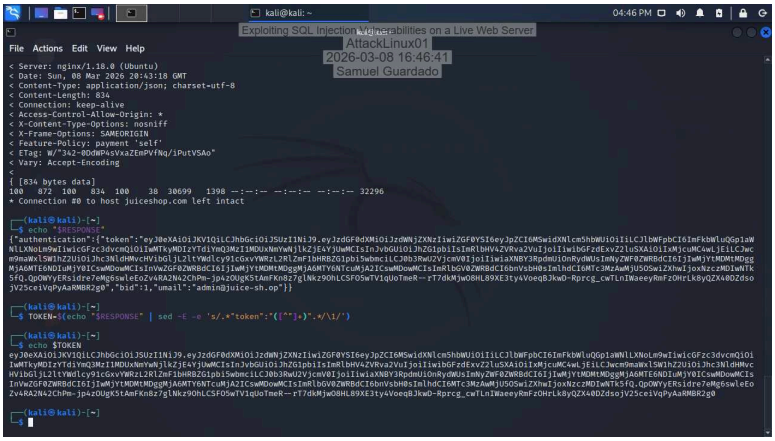
30. **Make a screen capture** showing your successful exploitation with the Christmas Super Surprise Box in your shopping basket.



Section 2: Applied Learning

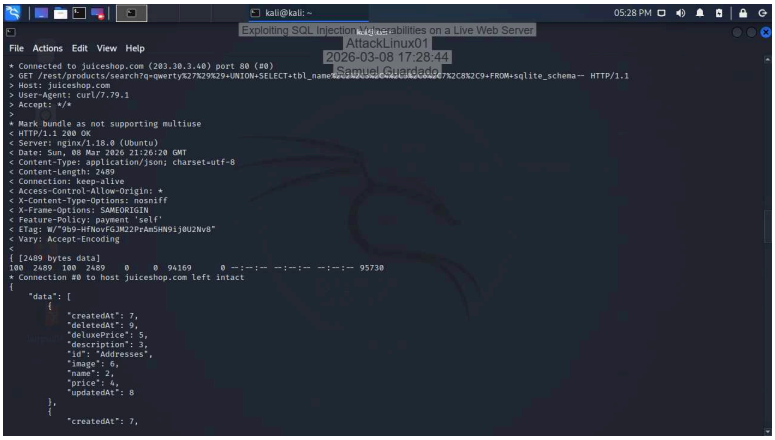
Part 1: Use curl to exploit the login page

11. **Make a screen capture** showing the login token.

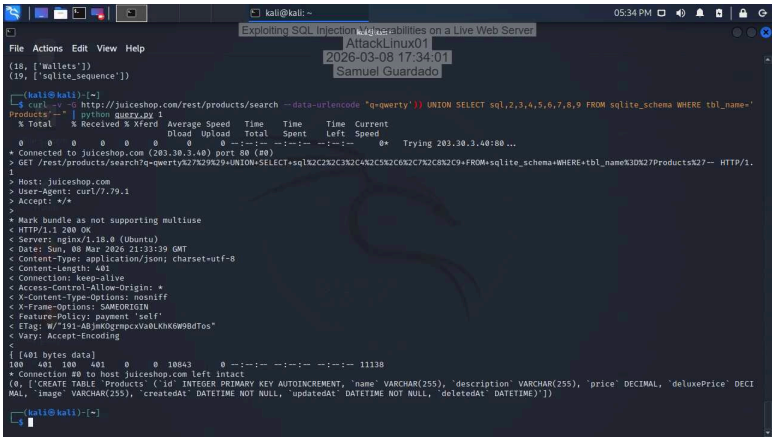


Part 2: Use curl to examine the database

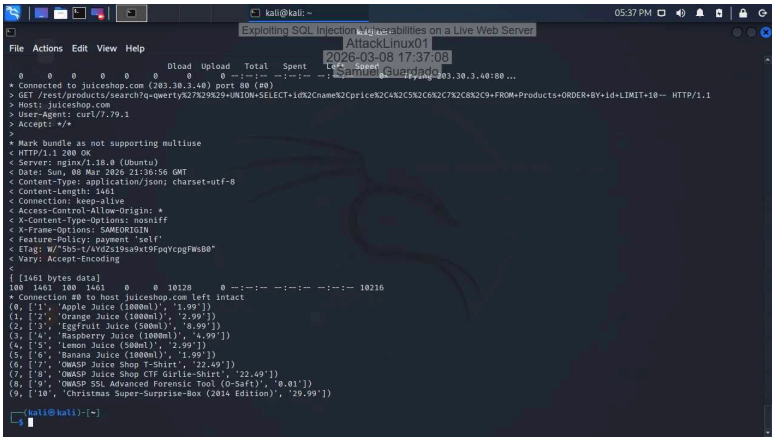
3. **Make a screen capture** showing the first item in the data field of the response.



6. **Make a screen capture** showing the SQL statement for the products table.



8. **Make a screen capture** showing the 10 products from the response.



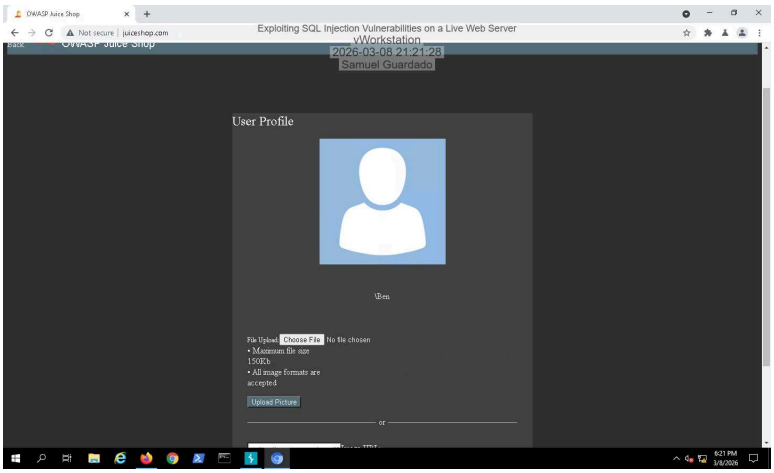
Section 3: Challenge and Analysis

Part 1: User Login Challenge

1. **Document the exploit** that you used to log in as Bender.

First, I sent a failed attempt login. Then I went to the HTTP history to find the failed login request. After finding it, I sent it to the repeater, and used the following injection. "email": "bender@juice-sh.op'--" "password": "x" then I went back to the juice shop website and used these credentials. After that, I was able to log into the account and change the name.

2. **Make a screen capture**showing Bender's account page with the username displayed under the avatar image (it should display *Ben*).

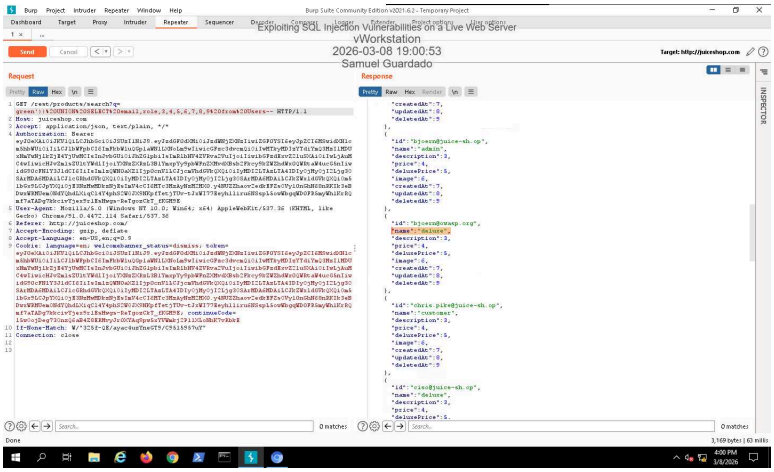


3. **Document key aspects of a plan**that you would recommend implementing to prevent this type of vulnerability.

To prevent this type of vulnerability it is extremely important to get rid of directly inserting user input to the SQL Query by using prepared statements instead. This prevents SQL injections from being executed by separating user input from SQL commands. Also, inputs should always be validated and sanitized, to assure only correct data is accepted.

Part 2: All Your Credentials

1. **Make a screen capture**showing the modified search in the Request panel and the JSON results with the user account information of a user whose role is *deluxe* in the Response panel.



2. **Document an explanation**for why the JSON object with the results in the Response pane shows id, name, description, price, etc., instead of email, role, etc.

In this database /rest/products/search returns product objects(ProductID, name, description, and price). The function "UNION SELECT" extracts data from the "users" table, and then it returns the same number of columns as the original product query (the objects in the products table). That's why when the "email" and "role" fields are returned, the fields in the JSON tab appear under the fields of "id" and "name."